React JS

Agenda

- **What is a Props? & Higher Order Component**
- **React hooks**
- **useState hook**

string | num | object | array

Props

Passing Props to a Component

React components use props to communicate with each other. Every parent component can pass some information to its child components by giving them props. Props might remind you of HTML attributes, but you can pass any JavaScript value through them, including objects, arrays, and functions.

```
<Avatar
  person={{ name: 'Lin Lanying', imageId: '1bX5QH6' }}
  size={100}
/>
```

Read props inside the child component

You can read the props in the child component function

```
function Avatar(props) {
  //props.person and props.size
}
```

Or you can use ES6 destruct option to read the props sent keys

```
function Avatar({person,size = 50}) {}
```

React Hooks

- React Hooks are simple JavaScript functions that we can use to isolate the reusable part from a functional component. Hooks can be stateful and can manage side effects

The Rules of React Hooks

- Only Call Hooks at the Top Level
- Only Call Hooks from React Functions

React Hooks

- A hook is a special function that lets you "hook into" various React features. Imagine a function that returns an array with two values. That’s what

useState Hook

- **The first value:** a variable with the state.
- **The second value**: a variable with an handler (a function to change the current state).

useState Hook

To manage states. Returns a stateful value and an updater function to update it.

State lets a component “remember” information like user input. For example, a form component can use state to store the input value.

We use the array destructure, as we know its first and second position values, and we know they correspond to the state value and to a handler to change it.

Usage :

```
const [state, setState] = useState(initialStateValue)
```

https://www.freecodecamp.org/news/learn-react-usestate-hook-in-10-minutes/

Hazem Alsaqaan

```
function App() {
  let state = 1

 const increaseHandler = ()=>{
  state += 1
  console.log(state)
 }
  return (
    <div>
      <p>{state}</p>
      <button onClick={increaseHandler} >increase</button>
    </div>
  )
}

export default App
```

Without

useState( )

React doesn’t know it needs to re-render.

React only re-renders the component when state managed by useState changes.

Hazem Alsaqaan

```react/usestate.jsx
import React, { useState } from 'react'

function App() {
  const [num, setNum] = useState(1)

 const increaseNumHandler = ()=>{
  // setNum(num + 1)
  setNum((prev)=> prev + 1)
}
console.log(num)
  return (
    <div>
      <p>{num}</p>
      <button onClick={increaseNumHandler} >increase</button>
    </div>
  )
}

export default App
```

With

useState( )

useState creates a state variable that triggers re-rendering when it changes.

useState( )With String

Hazem Alsaqaan

```
1  import React, { useState } from 'react'
2
3  function App() {
4    const [title, setTitle] = useState("Please Login..")
5
6   const updateTitleHandler = ()=>{
7    setTitle("Hello ITI Member")
8  }
9  console.log(title)
10   return (
11     <div>
12       <p>{title}</p>
13       <button onClick={updateTitleHandler} >Update</button>
14     </div>
15   )
16 }
17
18 export default App
```

useState( )With Object

Hazem Alsaqaan

```
1  import React, { useState } from 'react'
2
3  function App() {
4    const [currentUser, setCurrentUser] = useState({
5      username: "empty",
6      email: "empty"
7    })
8
9   const updateCurrentUserHandler = ()=>{
10    // setCurrentUser({username: "hazem"})
11    // setCurrentUser({email: "hazem@gmail.com"})
12    setCurrentUser({...currentUser, username: "hazem", email: "hazem@gmail.com"})
13 }
14 console.log(currentUser.username, currentUser.email)
15
16   return (
17     <div>
18       <p>{currentUser.username}</p>
19       <p>{currentUser.email}</p>
20       <button onClick={updateCurrentUserHandler} >Update user</button>
21     </div>
22   )
23 }
24
25 export default App
```

useState( )With Array

Hazem Alsaqaan

```
import React, { useState } from 'react'

const initialValues = [
  {
    id: "1",
    productName: "pro 1"
  },
  {
    id: "2",
    productName: "pro 2"
  },
]
function App() {
  const [products, setProducts] = useState(initialValues)

 const addToProductsHandler = ()=>{
  setProducts([...products, {id: "3", productName: "pro 3"}])
}
console.log(products)

  return (
    <div>
      {
        products.map((item)=>(
          <p key={item.id}>{item.productName}</p>
        ))
      }
      <button onClick={addToProductsHandler} >Add Product</button>
    </div>
  )
}

export default App
```

Rendering Lists

- You will often want to display multiple similar components from a collection of data. You can use the JavaScript array methods to manipulate an array of data like filter() and map() with React to filter and transform your array of data into an array of components.

- You need to give each array item a key — a string or a number that uniquely identifies it among other items in that array.

- Keys tell React which array item each component corresponds to, so that it can match them up later. This becomes important if your array items can move (e.g. due to sorting), get inserted, or get deleted. A well-chosen key helps React infer what exactly has happened, and make the correct updates to the DOM tree.

https://react.dev/learn/rendering-lists#why-does-react-need-keys

Conditional Rendering

Your components will often need to display different things depending on different conditions. In React, you can conditionally render JSX using JavaScript syntax like if statements, &&, and ? : operators.

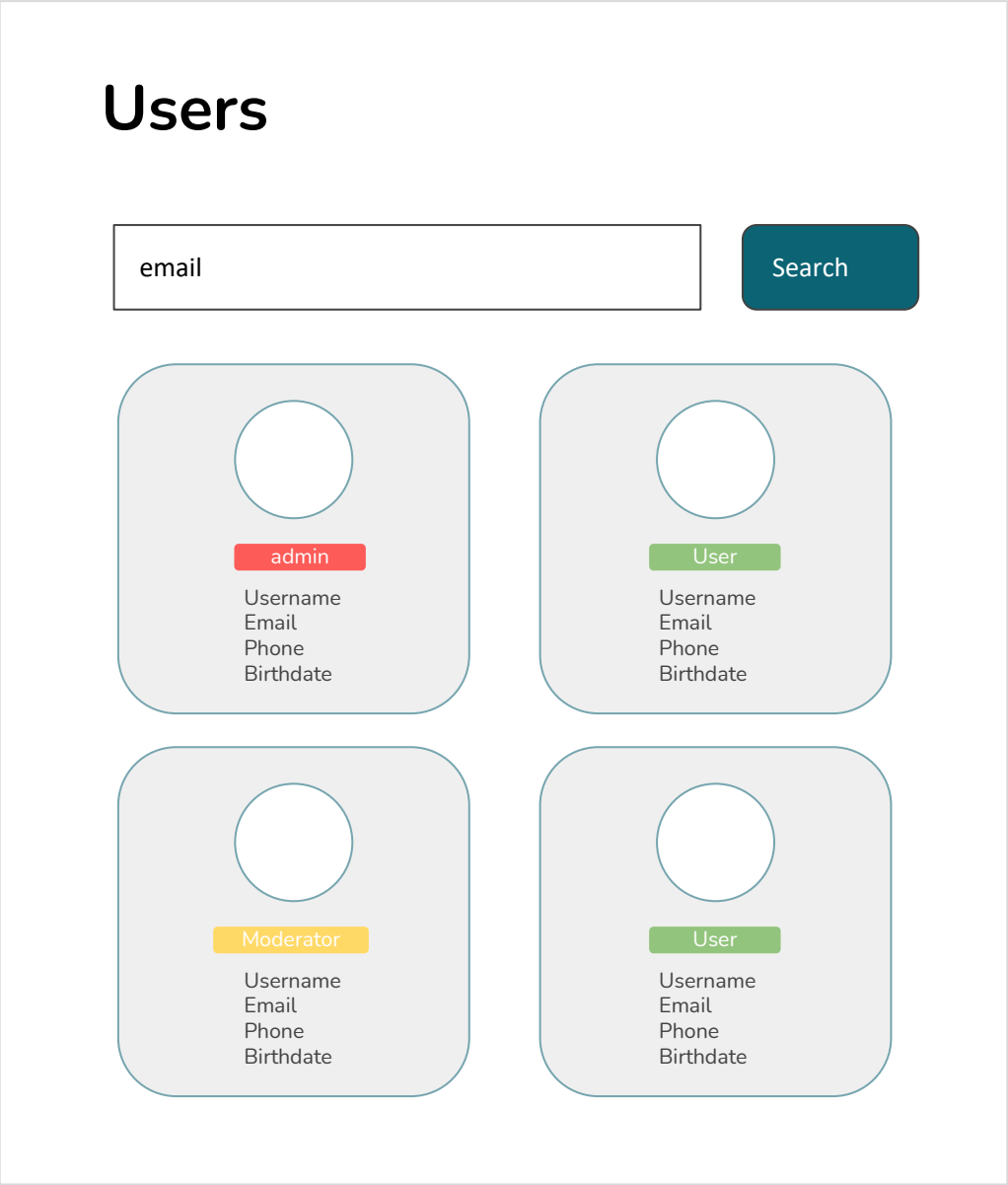
Thank You

Lap

Users list

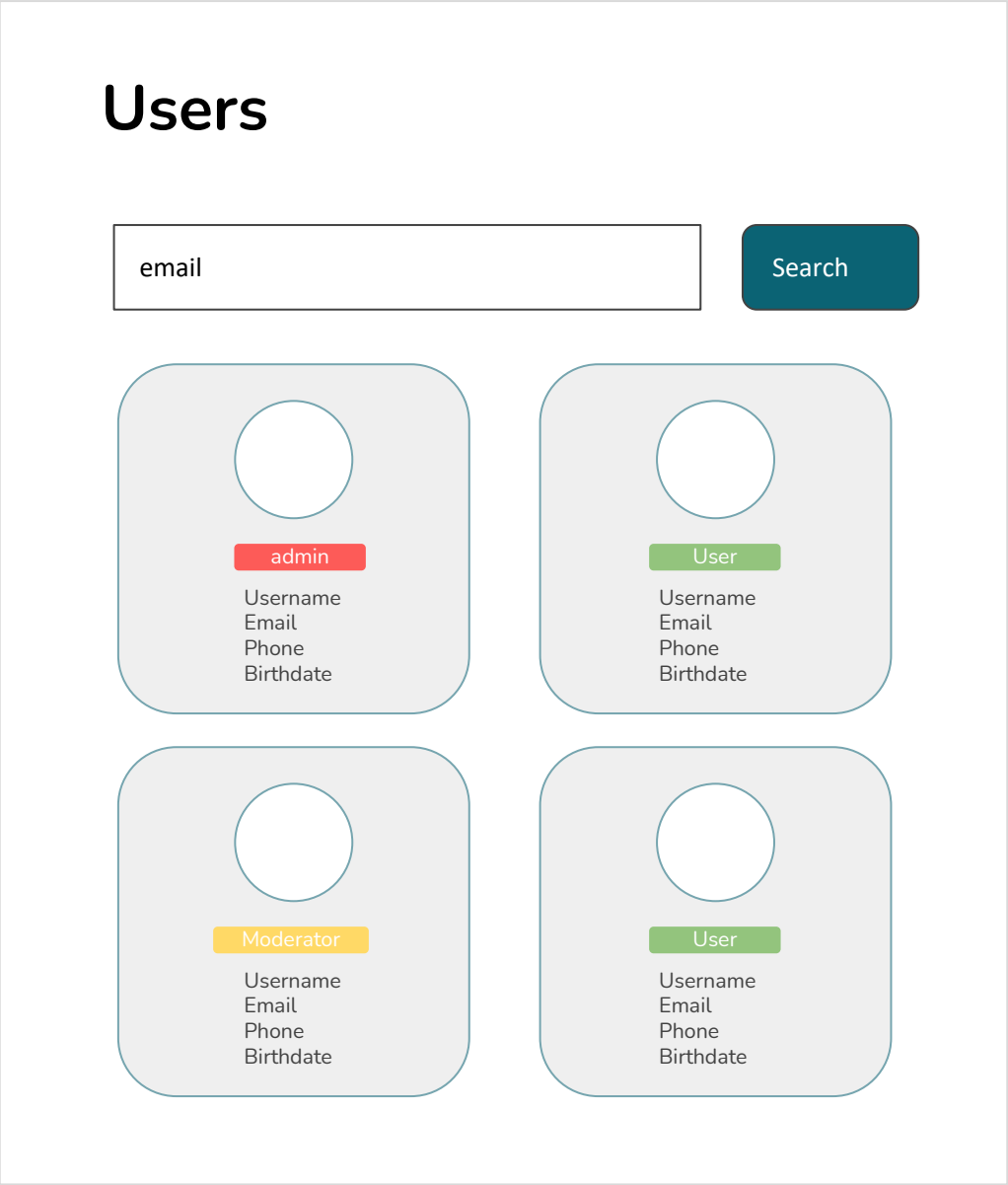
- Using the provided users list to render the users cards with the following:
  - Profile picture
  - Username
  - Email
  - Phone number
  - Birthdate
  - Role chip

- Based on the user role, show a chip with a different color. If admin shows chip with red, if user shows chip with green, if moderator show chip with yellow



Users list

- you can search and in the users list by email, and after user search, a reset button will appear when clicked, which will reset the fields and show all users again.



Task : To Do App

Create a to-do app to create self-notes with the following features:

- User can add new task
- The user can delete Task
- The user can mark as completed, and when
- marked as completed, it will be marked with a line through.

Using reusable components and passing data through components